

41052 Programming Assignment 1

A* Search, 15 Puzzle

Benjamin Hood - 25478064

Video walkthrough: <https://youtu.be/y8b-pxQh2hs>

(a) What I Built

The algorithm I built was A* search. A* is one of the most widely used techniques for graph traversal and pathfinding (GeeksforGeeks, 2016). I chose it because, despite being familiar with Breadth-First Search (BFS) and Dijkstra's algorithm, I had never implemented A* itself, which combines ideas from both. I implemented it in C++ for its speed and manual memory control, which suited algorithms practice well.

A* extends Dijkstra's algorithm by using a heuristic to guide which state is expanded next. Each state is scored as $f(n) = g(n) + h(n)$, where $g(n)$ is the accumulated cost to reach it and $h(n)$ is a heuristic estimate of the cost remaining; the state with the lowest $f(n)$ is always expanded next. Provided $h(n)$ never overestimates the true remaining cost (an admissible heuristic), A* is guaranteed to find an optimal solution, while typically exploring far fewer states than Dijkstra's or BFS, since the heuristic actively steers the search toward the goal rather than expanding uniformly outward.

My implementation represents each search state generically, in the 15 puzzle, a board configuration, though the search logic itself has no fixed notion of what a state actually is. States are ordered by a priority queue keyed on $f(n)$. A closed set (a hash set of serialized states) prevents the same state from being expanded more than once, which matters because many different move sequences can reach the same state; without this the search would repeat large amounts of work. I also added parent pointers for each discovered state, so the full sequence of states from start to goal can be reconstructed afterwards, as a move count alone says nothing about how the goal was reached.

(b) The Tool

To test and demonstrate the A* implementation, I applied it to the 15 puzzle. The 15 puzzle is a 4x4 sliding tile puzzle where the goal is to arrange tiles 1-15 in order around a single blank space, using only slides of a tile into the adjacent blank. In A*, it is represented by a graph, where each board configuration is a state, each move is an edge, and the heuristic used is the Manhattan distance, which measures how far each tile is from its goal position by summing the absolute difference between their coordinates (GeeksforGeeks, 2025).

To make the puzzle interactive, I built a command-line tool that takes a board and solves it. It accepts input in two ways: an explicit board representing by 16 space-separated values, with 0 for the blank square (e.g., `--board "1 2 3 4 5 6 7 8 9 10 11 0 13 14 15 12"`), or a randomly generated one (`-scramble N --seed S`), which starts from the goal state and makes N random legal moves. I added the scramble option because writing a 16-number board is

tedious and easy to make errors, while scrambling from the goal guarantees the result is solvable and allows the complexity to be controlled by varying N.

Inside the tool, `solve()` runs the A* loop described in part (a) and returns both the move count and the full solution path, e.g. `--scramble 20 --seed 7` resolves to an 18-move solution.

(c) What I learned

At first, this project seemed quite challenging. To learn how A* works and understand the 15-puzzle, I watched the video provided on the topic selection page. I then started with LeetCode 773, which is a smaller 2x3 version of the puzzle. After building and testing my solution, I adapted it to work with the larger 4x4 15-puzzle.

Something I did not initially consider, was that moving from a 2x3 board to a 4x4 board introduced the challenge of some puzzles being impossible to solve. Without checking this first, A* could spend a very long time searching before determining that no solution exists. This meant I had to implement a check to see if a board was solvable.

One thing that I learned was how the Manhattan distance heuristic was calculated. When trying to implement this on my own, it was harder than expected, because I was not sure how to find the goal state efficiently for the current tile. I overlooked the idea of using simple division and modulo by the number of columns to find each tile number's correct location, which could be compared with its current location to calculate the heuristic.

(d) AI Use

I used ChatGPT for direction and small bug fixes early in my A* implementation, and Claude more extensively for generalising the implementation to the 4x4 board, restructuring the code into separate files, debugging, and drafting parts of this report and the accompanying README.

An example of AI being unhelpful was when using Claude to draft the solution path output. The code outputs the initial board, before outputting the steps that the algorithm took, starting at step 0 (step 0, step 1, step 2...). This meant the same board was printed twice in a row, as the starting board and then step 0. Step 0 was redundant and confusing, so I changed it so that the steps start at 1.

Another example was when implementing my code on LeetCode I was facing some issues with test cases, indicating that there was something wrong that I was overlooking. I checked with Claude, and despite identifying some minor bugs, it did not catch the major issue that was causing me problems. The problem was my goal state was defined incorrectly, as {0, 1, 2} {3, 4, 5}, instead of {1,2,3}, {4,5,0}, so it was incorrectly calculating the heuristic as all 6 numbers were in the wrong position. To overcome this, I put samples of the failed test cases in, and it was able to identify the simple error for me to fix.

Much of this implementation follows the same structure as other graph-traversal algorithms I've studied, so I was reasonably confident in most sections, including ones AI helped draft. One section I initially took on trust was `serialize()`, as I didn't understand why it was

necessary and was confused when an earlier version without it didn't work, until I understood it exists purely to convert a board into a hashable string key, since there's no built-in hash for `vector<vector<int>>`. `randomScramble()` was also mostly designed by AI rather than something I designed myself, though I understand how it works: it starts at the goal state and applies N random legal moves, which guarantees a solvable result without needing to separately verify solvability.

References

GeeksforGeeks. (2016, June 16). *A* search algorithm*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/a-search-algorithm/>

GeeksforGeeks. (2025, August 26). *Manhattan distance*. GeeksforGeeks.

<https://www.geeksforgeeks.org/data-science/manhattan-distance/>